
Model Extraction (Model Stealing) Attack

Stealing the Functionality of a Black-Box Image Classifier

Submitted by

Abrar Zahin Raihan (ID: 2105009)

Md. Mahmud Hasan (ID: 2105027)

1 Overview of the Project Idea

Model extraction, also known as model stealing, is an attack in which an adversary with only *black-box query access* to a trained model reconstructs a functionally equivalent copy [1–3]. The attacker never sees the victim’s weights, architecture, or training data, and observes only what the model returns for chosen inputs [1, 2]. Throughout this report we write f_V for the victim model and f_S for the substitute, the copy that we train.

Two success criteria are worth naming, because the reference papers distinguish between them:

- **Accuracy:** the stolen model performs the task well on real data (Jagielski et al. [3]).
- **Fidelity:** the stolen model *agrees with the victim*, reproducing even its mistakes (Jagielski et al. [3]).

Our target is to closely reproduce **functionality stealing** in the style of *Knockoff Nets* (Orekondu et al. [2]): we query the victim with images from any convenient pool, treat the victim’s returned predictions as labels, and train our own model on those (input, prediction) pairs. Crucially, we need **neither the victim’s training data nor its architecture** [2]. Tramèr et al. [1] showed that the same idea works even against simple machine-learning-as-a-service models, sometimes by direct equation solving; Knockoff Nets extends it to deep networks through a query-and-imitate procedure [2]. The same substitute-training primitive had earlier been used by Papernot et al. [4] as a stepping stone to black-box evasion attacks.

The core claim we will demonstrate is that *a black-box classifier can be duplicated cheaply, purely through its prediction API* [1, 2].

2 Network Topology, Components, and Timing Diagram

The network here is minimal. Because this is an application-layer attack, the topology is just a query channel between two components. The adversary we assume, which has black-box access and no knowledge of the victim’s parameters or training data, is the standard model-extraction threat model of Tramèr et al. [1], placed within the taxonomy of adversary goals, knowledge, and capabilities set out by Jagielski et al. [3]. Figure 1 shows the arrangement.

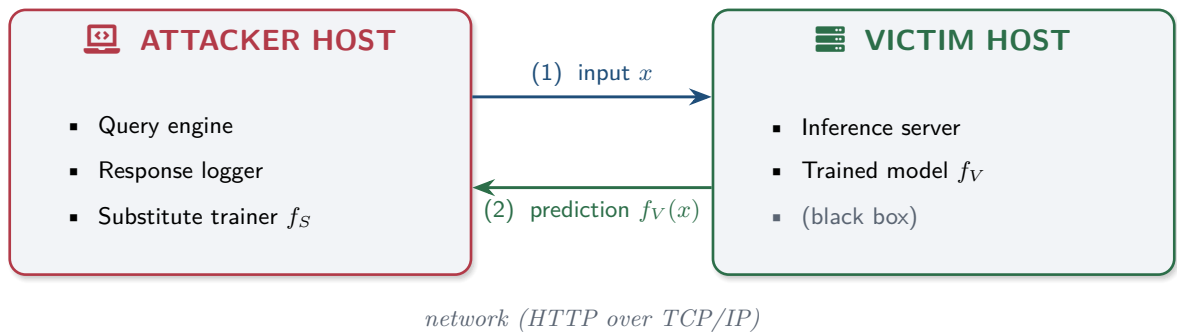


Figure 1: Attack topology. (1) The attacker sends a chosen input x to the victim’s prediction API. (2) The victim returns its prediction $f_V(x)$, a label or a probability vector. The attacker treats each $(x, f_V(x))$ pair as a training example for f_S .

Both roles can live on **one machine** (the victim as a local server on `localhost` and the attacker as a script), or on **two hosts on the same LAN**. Nothing about the attack depends on the transport.

2.1 Components

The five components that together realise the attack are listed in Table 1.

Component	Role
Victim model f_V	Pre-trained classifier the attacker cannot inspect.
Inference server	Wraps f_V , accepts inputs, returns predictions.
Attacker query engine	Selects inputs and sends them within a query budget.
Response logger	Stores every (input, prediction) pair.
Substitute trainer	Trains the stolen model f_S on the logged pairs.

Table 1: Components of the attack and the role each one plays.

2.2 Timing Diagram (query loop)

Under normal operation the prediction API serves a single request: an honest client sends one input and consumes one answer. The attack reuses exactly the same request and response messages but changes the intent behind them. The attacker issues inputs in batches, logs every returned prediction, and repeats until the query budget is spent; only then does it move offline to train the substitute, with no further queries.

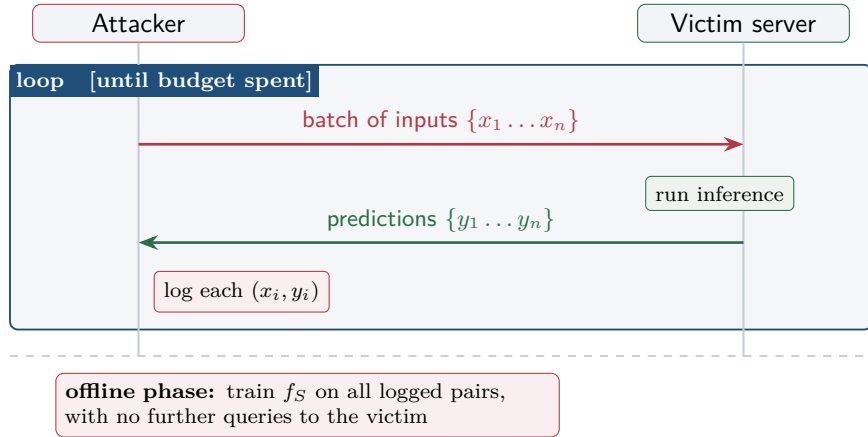


Figure 2: Attack timing diagram. The attacker repeatedly sends batches of inputs and logs the returned predictions until the query budget is exhausted, then trains the substitute f_S entirely offline.

3 Packet / Message Structure

This attack operates at the **application layer**, so there is no custom L2/L3 frame to craft. The L2, L3, and L4 headers are ordinary Ethernet, IP, and TCP, and they are not part of the attack. What matters is the **request/response message** carrying the model input and output. Figure 3 places that payload inside the surrounding protocol stack.

Request (attacker → victim):

```

POST /predict HTTP/1.1
Content-Type: application/json

{ "input": "<base64-encoded image tensor>" }
```

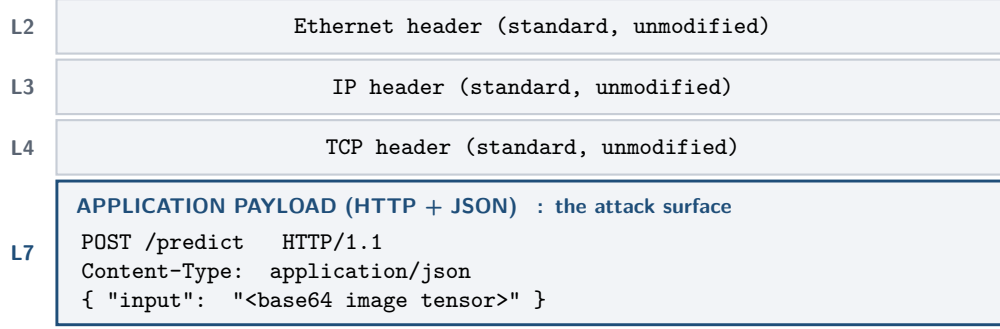


Figure 3: Protocol stack for one query. The lower layers are ordinary and are left untouched; the entire attack lives in the application-layer payload.

Response (victim $\rightarrow$ attacker):

```
HTTP/1.1 200 OK
Content-Type: application/json

{ "label": 3,
  "probabilities": [0.02, 0.05, 0.11, 0.71, ...] } // may be present
or withheld
```

The single most important field for the attacker is the **output granularity** [1, 3]:

- **Full probability vector:** the richest signal, needing the fewest queries. The soft outputs carry dark knowledge in the distillation sense [5], and Knockoff Nets trains directly on them [2].
- **Top-1 label only:** a weaker signal, needing more queries, but the attack still works. Tramèr et al. [1] showed that omitting confidence values is not by itself a sufficient countermeasure, and Jagielski et al. [3] treat label-only access as an access restriction rather than a barrier.

This distinction is exactly what the defense in Section 6 exploits.

4 Implementation Plan

1. **Building the victim.** We will train a small CNN (for example a ResNet-18 [10] or a four-layer CNN) on CIFAR-10 [9] and save the weights. We will wrap it in a minimal Flask or FastAPI server that returns predictions. For the rest of the steps it is treated as a black box.
2. **Choosing a query set (the transfer set).** We will pick a pool of images to query with, and we will compare two options:
 - *Same domain* (main idea): CIFAR-10 test images or a held-out slice.
 - *Different domain* (alternative, Knockoff Nets [2]): unrelated images, for example a subset of CIFAR-100 [9], which shares the same 32×32 format as CIFAR-10 but whose classes barely overlap. This proves that we do not need the victim’s data, the central observation of [2], and it was independently reported for non-problem-domain data by Correia-Silva et al. [8].
3. **Query and log.** We will send inputs in batches and record each returned prediction. We will also fix a query budget (for example 5k, 10k, or 20k) so that we can plot cost against quality; budget-versus-quality curves are the standard way that [2, 3] report extraction cost.

4. **Train the substitute f_S .** This step is interesting. We will try out different architectures, since Knockoff Nets shows that the knockoff need not match the victim’s architecture [2]. We will train on (query image, victim prediction) pairs under two strategies:
 - i. Considering that prediction probabilities are available: a soft-label or KL loss in the knowledge-distillation style [5].
 - ii. Considering that only labels are available: standard cross-entropy on the hard labels, the label-only setting of [1, 3].
5. **Evaluating two metrics on a clean test set.**
 - **Accuracy** of f_S on the true task [3].
 - **Agreement / Fidelity**, the fraction of test inputs where $f_S(x) = f_V(x)$, following the accuracy and fidelity split of Jagielski et al. [3].
6. **Sweeping the budget.** We will repeat steps three through five at each budget and plot *accuracy and fidelity against the number of queries*. This single plot is the headline result of the project.

Stack: Python, PyTorch, torchvision, FastAPI.

5 Expected Outcome of a Successful Attack

- The substitute f_S reaches **high test accuracy**, typically approaching the victim’s accuracy, using far fewer queries than the victim’s original training set size, as reported in [2] and [3].
- **Agreement with the victim** rises with the query budget and plateaus, demonstrating high fidelity even on inputs for which f_S was never explicitly told the correct answer. Jagielski et al. [3] also argue analytically why a learning-based attack plateaus short of true functional equivalence.
- The attack **succeeds without the victim’s architecture or training data** [2], showing that exposing a prediction API can leak the model itself [1].
- The accuracy-versus-queries curve gives a concrete, quantitative story: *with N queries, an adversary recovers $X\%$ of the victim’s functionality*.

Our expected story line is f_S within a few percentage points of f_V at a modest query budget, using a different architecture and out-of-distribution query data, which is the regime in which Knockoff Nets reports its strongest results [2].

6 Defense Idea

Main defense: reduce output granularity. The victim returns the **top-1 label only**, or rounds or truncates the probabilities. This removes the soft-label signal and measurably increases the query budget the attacker needs. Rounding confidence values is exactly the countermeasure evaluated by Tramèr et al. [1], who caution that it raises the cost without eliminating the attack, and [3] likewise treats output granularity as an access restriction on the adversary. We will try to measure the effect via the budget curve.

Alternatives.

1. **Rate limiting and query quotas.** Cap queries per client per time window, making large budgets expensive or slow [3, 6].

2. **Query-distribution monitoring (PRADA-style).** Extraction attacks often submit inputs whose distribution differs from normal traffic, for example inputs that are out of distribution or highly diverse across a batch. We can monitor the distribution of incoming queries and flag anomalous patterns; PRADA (Juuti et al. [6]) detects extraction from the distribution of pairwise distances between successive queries.
3. **Output perturbation.** Add small, controlled noise to the returned probabilities to poison the substitute’s training signal while keeping the predictions useful to honest users, which is the prediction-poisoning defense of Orekondy et al. [7].

References

- [1] F. Tramèr, F. Zhang, A. Juels, M. K. Reiter, and T. Ristenpart, “Stealing Machine Learning Models via Prediction APIs,” in *Proc. 25th USENIX Security Symposium (USENIX Security ’16)*, Austin, TX, USA, Aug. 2016, pp. 601–618.
- [2] T. Orekondy, B. Schiele, and M. Fritz, “Knockoff Nets: Stealing Functionality of Black-Box Models,” in *Proc. IEEE/CVF Conf. on Computer Vision and Pattern Recognition (CVPR)*, Long Beach, CA, USA, Jun. 2019, pp. 4954–4963.
- [3] M. Jagielski, N. Carlini, D. Berthelot, A. Kurakin, and N. Papernot, “High Accuracy and High Fidelity Extraction of Neural Networks,” in *Proc. 29th USENIX Security Symposium (USENIX Security ’20)*, Aug. 2020, pp. 1345–1362.
- [4] N. Papernot, P. McDaniel, I. Goodfellow, S. Jha, Z. B. Celik, and A. Swami, “Practical Black-Box Attacks against Machine Learning,” in *Proc. ACM Asia Conf. on Computer and Communications Security (ASIA CCS ’17)*, Abu Dhabi, UAE, Apr. 2017, pp. 506–519.
- [5] G. Hinton, O. Vinyals, and J. Dean, “Distilling the Knowledge in a Neural Network,” in *NIPS 2014 Deep Learning Workshop*, 2015, arXiv:1503.02531.
- [6] M. Juuti, S. Szyller, S. Marchal, and N. Asokan, “PRADA: Protecting Against DNN Model Stealing Attacks,” in *Proc. IEEE European Symposium on Security and Privacy (EuroS&P)*, Stockholm, Sweden, Jun. 2019, pp. 512–527.
- [7] T. Orekondy, B. Schiele, and M. Fritz, “Prediction Poisoning: Towards Defenses Against DNN Model Stealing Attacks,” in *Proc. International Conference on Learning Representations (ICLR)*, Apr. 2020.
- [8] J. R. Correia-Silva, R. F. Berriel, C. Badue, A. F. De Souza, and T. Oliveira-Santos, “Copycat CNN: Stealing Knowledge by Persuading Confession with Random Non-Labeled Data,” in *Proc. International Joint Conference on Neural Networks (IJCNN)*, Rio de Janeiro, Brazil, Jul. 2018, pp. 1–8.
- [9] A. Krizhevsky, “Learning Multiple Layers of Features from Tiny Images,” Technical Report, Department of Computer Science, University of Toronto, 2009.
- [10] K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” in *Proc. IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA, Jun. 2016, pp. 770–778.